Adaptive Traffic Signal Control Using Traffic-Demand and Waiting-Time Analysis

A B.Tech Minor Project (Batch B1) — Department of Computer Science

What problem this solves

Most urban traffic signals run on fixed timers — a preset number of seconds for each direction, regardless of how much traffic is actually there. This wastes green time on empty approaches and lets busy ones sit in unnecessary queues. This project builds a software controller that watches real-time traffic conditions at an intersection and dynamically decides how long each direction should get green light, instead of using a fixed schedule.

A key design goal beyond just "clear the busiest queue first": the controller must guarantee that low-traffic approaches are never left waiting indefinitely just because another direction is busier. Fairness is treated as a first-class, provable requirement — not an afterthought.

How it's being built

The system is developed and tested entirely in **SUMO (Simulation of Urban Mobility)**, a traffic simulation environment, controlled via **Python** using **TraCI/libsumo** to communicate with the simulator in real time. Simulation is used deliberately — testing signal-control logic on a real intersection isn't safe or legal for a student project, but SUMO makes it possible to run thousands of traffic scenarios cheaply and evaluate the algorithm before any real-world consideration. This is the same approach used in professional traffic-engineering research, not a compromise unique to a student project.

The adaptive controller is benchmarked against a conventional fixed-time signal controller under identical simulated traffic conditions, using four metrics: average waiting time, queue length, travel time, and vehicle throughput.

Current status (as of September 2026)

Synopsis submitted (dated 24 Aug 2026) — project title, team, and scope formally registered.
Core control algorithm: not yet finalized. Candidates under evaluation include max-pressure control, a custom weighted-priority heuristic, Webster's-formula-based dynamic cycling, fuzzy-logic control, and reinforcement learning — being scored against implementation time, team skill fit, and how explainable the result will be. Reinforcement learning is currently considered unlikely to be chosen given the project timeline.
Fairness mechanism: decided in principle, not yet implemented. The project will guarantee a *hard bound* — no approach ever waits longer than a defined maximum time — rather than a vague statistical fairness claim. This bound will be mathematically derived (using an "aging" term that guarantees a starved approach's priority eventually overtakes any busier approach) and then empirically stress-tested with imbalanced and adversarial traffic scenarios to confirm the bound holds in practice.

Baseline fixed-time controller + SUMO metrics pipeline: not yet built. This is the next concrete engineering step — it's required regardless of which algorithm is ultimately chosen, and it will supply real numbers to calibrate the target improvement against.
Target improvement figure: not yet fixed. Rather than committing to a number in advance, the plan is to run the fixed-time baseline first, use published literature (typically ~10–30% average-wait reduction for adaptive vs. fixed control) as a sanity check, and only then set a specific, data-backed target.
Real-data grounding: planned, not yet sourced. Rather than relying purely on synthetic simulated demand, the team intends to calibrate SUMO traffic demand using either real traffic data or manual on-site vehicle counts at an actual Mysuru intersection, since generic public datasets available online largely reflect non-Indian traffic patterns.

Planned methodology (short version)

Build and validate the fixed-time baseline simulation and full metrics logging in SUMO.
Timebox a short prototype of the top 1–2 algorithm candidates on the same test intersection to compare real implementation effort, not just theory.
Finalize the control algorithm and the fairness weighting scheme, deriving the maximum-wait guarantee from physical constraints (minimum green time, signal clearance/amber time — per IRC:93 guidelines), the number of approaches, and standard traffic-engineering cycle-length practice (60–120 seconds).
Validate fairness and performance across multiple random seeds, several traffic-demand regimes (light/moderate/heavy/imbalanced), and deliberately adversarial stress scenarios.
Calibrate simulated demand to a real intersection where possible.
Get the algorithm, fairness mechanism, and target metric formally reviewed and signed off by the internal guide before full-scale implementation.

Future work (explicitly out of current scope)

The current phase is simulation-only. A real-world deployment path has been thought through conceptually: existing CCTV cameras or traffic sensors would supply live footage, a computer-vision system (e.g. YOLO) would estimate vehicle counts and queue lengths from that footage, and this real-time traffic-state data would feed the same adaptive control algorithm developed in simulation. The algorithm's decisions would then need to be integrated with a city's existing traffic-signal infrastructure through an authorized traffic-management system, under supervision of the relevant traffic authority. This is treated as a future extension, not a deliverable of the current project.

Team

Dewanshi Saran Goel
Hitha Patel M A
Janapriya M J
Likitha R

Internal Guide: Dr. Shabana Sultana, Professor

Context

This is a competitive academic setting (this project is being evaluated alongside roughly 150 other teams). "Adaptive traffic signal control in SUMO" is a fairly common project topic, so the team's differentiation strategy is deliberately layered on top of the core idea rather than a topic change: grounding the simulation in real traffic data, making the fairness claim a rigorously provable guarantee rather than a soft claim, and using statistically rigorous evaluation (multiple seeds, varied load conditions, adversarial testing) — areas where most comparable student projects tend to be shallow.